

# 생성형 **AI Web App** 기획 및 개발 프로젝트 수행 체크리스트

## 프로젝트 전체 수행 흐름

문제 정의 및 기획 → 요구사항 정의 → UX/UI 설계 → 시스템 설계 → 개발환경 구축 → 데이터 준비 → AI 모델링 → Web 개발 → 생성형 AI 구현 → 통합 → 테스트 → 배포 → 고도화 → 발표

## 1. 문제 정의 및 **AI Web App** 서비스 기획

### 수행 체크리스트

- ☐ 산업·업무 현장에서 해결하고자 하는 문제를 선정하였다.
- ☐ 문제의 발생 원인과 현재 해결 방식의 문제점을 분석하였다.
- ☐ 해당 문제의 대상 사용자(**Target User**)를 정의하였다.
- ☐ 사용자 **Pain Point**를 도출하였다.
- ☐ 사용자 요구사항을 조사·정리하였다.
- ☐ AI를 활용하여 해결할 수 있는 부분을 도출하였다.
- ☐ 서비스 아이디어를 여러 가지로 도출하고 비교하였다.
- ☐ 최종 서비스 아이디어를 선정하였다.
- ☐ 서비스의 핵심 가치(**Value Proposition**)를 정의하였다.
- ☐ 서비스의 목표와 기대효과를 정의하였다.
- ☐ 핵심 기능과 부가 기능을 구분하였다.
- ☐ MVP(**Minimum Viable Product**) 범위를 정의하였다.
- ☐ 프로젝트 개발 일정과 단계별 목표를 수립하였다.
- ☐ 팀원별 역할과 담당 업무를 정의하였다.

### 활용 도구

**Notion, FigJam, Jira, Trello, Google Docs/Sheets**

### 주요 산출물

문제정의서, 서비스 기획서, **Persona, User Journey, MVP** 정의서, 프로젝트 계획서

## 2. 요구사항 분석 및 서비스 기능 정의

### 수행 체크리스트

- ☐ 서비스의 기능 요구사항을 정의하였다.
- ☐ 서비스의 비기능 요구사항을 정의하였다.
- ☐ 사용자 **User Story**를 작성하였다.
- ☐ 주요 **Use Case**를 정의하였다.
- ☐ 사용자 입력과 시스템 출력을 정의하였다.
- ☐ AI가 수행할 기능과 일반 **Web** 기능을 구분하였다.
- ☐ AI 모델이 필요한 기능을 정의하였다.
- ☐ LLM API가 필요한 기능을 정의하였다.
- ☐ RAG가 필요한 기능을 정의하였다.
- ☐ 일반 **CRUD**가 필요한 기능을 정의하였다.
- ☐ 기능별 우선순위를 결정하였다.
- ☐ MVP에 포함할 기능과 제외할 기능을 결정하였다.

### 활용 도구

**Notion, Jira, Miro, FigJam, Draw.io**

### 주요 산출물

요구사항 정의서, 기능 목록, **User Story**, **Use Case**, 기능 우선순위표

### 3. UX/UI 설계 및 **Prototype** 제작

#### 수행 체크리스트

- ☐ **Sitemap**을 작성하였다.
- ☐ 사용자 화면 흐름(**User Flow**)을 설계하였다.
- ☐ 주요 화면의 **Wireframe**을 제작하였다.
- ☐ 로그인/회원가입 등 사용자 흐름을 설계하였다.
- ☐ **AI** 입력 화면을 설계하였다.
- ☐ **AI** 결과 출력 화면을 설계하였다.
- ☐ 오류 및 예외 상황에 대한 화면을 설계하였다.
- ☐ 반응형 화면을 고려하였다.
- ☐ 주요 화면의 **Prototype**을 제작하였다.
- ☐ 팀원 또는 사용자에게 **Prototype**을 테스트하였다.
- ☐ 피드백을 반영하여 **UI/UX**를 수정하였다.

#### 활용 도구 및 기술

**Figma, FigJam, React, HTML, CSS, JavaScript**

#### 주요 산출물

**Sitemap, User Flow, Wireframe, UI Design, Prototype**

## 4. AI Web App 시스템 아키텍처 설계

### 수행 체크리스트

- ☐ 전체 시스템 구성도를 작성하였다.
- ☐ Front-end 구조를 정의하였다.
- ☐ Back-end 구조를 정의하였다.
- ☐ AI Server 구조를 정의하였다.
- ☐ Database 구성을 정의하였다.
- ☐ ML Model 구성 및 저장 위치를 정의하였다.
- ☐ Vector DB 구성을 정의하였다.
- ☐ LLM API 연동 구조를 정의하였다.
- ☐ Front-end → Back-end 데이터 흐름을 정의하였다.
- ☐ Back-end → AI Server 데이터 흐름을 정의하였다.
- ☐ AI Server → ML Model/RAG/LLM API 흐름을 정의하였다.
- ☐ Nginx Reverse Proxy 구조를 설계하였다.
- ☐ AWS EC2 배포 구조를 설계하였다.
- ☐ /, /api/, /ai/ 등 서비스 Routing 구조를 정의하였다.

### 활용 도구

**Draw.io, Lucidchart, Figma, Mermaid, AWS Architecture Diagram**

### 주요 산출물

시스템 아키텍처 구성도, 데이터 흐름도, 서비스 구조도

## 5. 개발환경 및 협업환경 구축

### 수행 체크리스트

- ☐ GitHub Repository를 생성하였다.
- ☐ 프로젝트 디렉터리 구조를 정의하였다.
- ☐ Python 개발환경을 구성하였다.
- ☐ Python Virtual Environment를 구성하였다.
- ☐ Node.js 및 npm 환경을 구성하였다.
- ☐ React/Vite 개발환경을 구성하였다.
- ☐ FastAPI 개발환경을 구성하였다.
- ☐ 필요한 Python Package를 정의하였다.
- ☐ Front-end Package를 정의하였다.
- ☐ .gitignore를 설정하였다.
- ☐ 환경변수 관리 방법을 정의하였다.
- ☐ Git Branch 전략을 정하였다.
- ☐ Commit 규칙을 정하였다.
- ☐ 팀원별 개발환경을 동일하게 구성하였다.

### 활용 기술 및 도구

**VS Code, Python, venv, Node.js, npm, Git, GitHub**

### 주요 산출물

**GitHub Repository, 개발환경 구성 문서, README**

## 6. 데이터 수집·분석 및 전처리

### 수행 체크리스트

- ☐ AI 모델 또는 서비스에 필요한 데이터를 정의하였다.
- ☐ 데이터 수집 방법을 결정하였다.
- ☐ 공공데이터/API/파일 등 데이터 출처를 확보하였다.
- ☐ 데이터의 저작권 및 활용 가능 여부를 확인하였다.
- ☐ 데이터를 수집하였다.
- ☐ 데이터 구조와 변수(**Feature**)를 분석하였다.
- ☐ **EDA**(탐색적 데이터 분석)를 수행하였다.
- ☐ 결측치를 처리하였다.
- ☐ 이상치를 처리하였다.
- ☐ 중복 데이터를 제거하였다.
- ☐ 필요한 **Feature**를 생성하였다.
- ☐ 데이터를 학습 가능한 형태로 변환하였다.
- ☐ **Train/Validation/Test** 데이터를 분리하였다.
- ☐ 전처리 과정을 재현할 수 있도록 정리하였다.

### 활용 기술 및 도구

**Python, Pandas, NumPy, Matplotlib, Jupyter Notebook, Google Colab**

### 주요 산출물

원천 데이터, 전처리 데이터, **EDA** 결과, 전처리 코드, 데이터셋

## 7. 인공지능 모델링 및 학습

### 수행 체크리스트

- ☐ AI 문제를 분류/회귀/예측 등의 형태로 정의하였다.
- ☐ 모델 입력값과 출력값을 정의하였다.
- ☐ 적용할 AI/ML 모델을 선정하였다.
- ☐ 모델 선정 이유를 정리하였다.
- ☐ 학습 데이터를 구성하였다.
- ☐ Feature Engineering을 수행하였다.
- ☐ AI 모델을 학습하였다.
- ☐ 학습 결과를 기록하였다.
- ☐ 적절한 평가 지표를 선정하였다.
- ☐ 모델 성능을 평가하였다.
- ☐ 여러 모델의 성능을 비교하였다.
- ☐ 필요할 경우 Hyperparameter를 조정하였다.
- ☐ 최종 모델을 선정하였다.
- ☐ 학습된 모델을 파일로 저장하였다.

### 활용 기술 및 도구

**Python, Scikit-learn, XGBoost, LightGBM, PyTorch/TensorFlow, Jupyter, Google Colab**

### 주요 산출물

학습 코드, 학습 모델, 모델 평가 결과, 모델 성능 비교표

## 8. AI 모델 서비스화 및 Prediction API 구현

### 수행 체크리스트

- ☐ 학습된 AI 모델을 AI Server에서 불러올 수 있도록 구성하였다.
- ☐ 입력 데이터 전처리 기능을 구현하였다.
- ☐ AI 모델 Prediction 기능을 구현하였다.
- ☐ FastAPI 기반 AI API Endpoint를 구현하였다.
- ☐ Request Schema를 정의하였다.
- ☐ Response Schema를 정의하였다.
- ☐ AI Prediction API를 테스트하였다.
- ☐ 오류 발생 시 예외처리를 구현하였다.
- ☐ API를 통해 AI 모델의 결과를 반환할 수 있도록 구성하였다.

### 활용 기술 및 도구

**Python, FastAPI, Uvicorn, Scikit-learn/PyTorch, Joblib/Pickle, Pydantic, Postman, Swagger**

### 주요 산출물

**AI Server, Prediction API, API 명세서, 모델 파일**



## 9. React 기반 Front-end 개발

### 수행 체크리스트

- ☐ Figma Prototype을 실제 Web 화면으로 구현하였다.
- ☐ React 프로젝트를 구성하였다.
- ☐ Component를 기능별로 분리하였다.
- ☐ 화면 및 페이지를 구현하였다.
- ☐ 사용자 입력 기능을 구현하였다.
- ☐ Form Validation을 구현하였다.
- ☐ API 호출 기능을 구현하였다.
- ☐ API 응답 결과를 화면에 표시하였다.
- ☐ Loading 상태를 구현하였다.
- ☐ 오류 메시지를 구현하였다.
- ☐ 반응형 화면을 구현하였다.
- ☐ 주요 사용자 시나리오를 실제로 수행할 수 있도록 구현하였다.

### 활용 기술 및 도구

**React, Vite, JavaScript/TypeScript, HTML, CSS, Axios/Fetch, VS Code**

### 주요 산출물

**React Web App, Front-end Source Code**

## 10. FastAPI 기반 Back-end 및 REST API 개발

### 수행 체크리스트

- ☐ Back-end 프로젝트 구조를 구성하였다.
- ☐ REST API Endpoint를 설계하였다.
- ☐ GET/POST/PUT/DELETE API를 구현하였다.
- ☐ Request/Response Schema를 정의하였다.
- ☐ Pydantic을 활용하여 데이터 검증을 구현하였다.
- ☐ 비즈니스 로직을 구현하였다.
- ☐ Front-end와 API를 연동하였다.
- ☐ API 오류 및 예외처리를 구현하였다.
- ☐ API 문서를 자동 생성하였다.
- ☐ Swagger에서 API를 테스트하였다.
- ☐ Postman에서 API를 테스트하였다.

### 활용 기술 및 도구

**Python, FastAPI, Uvicorn, Pydantic, REST API, OpenAPI, Swagger, Postman**

### 주요 산출물

**Back-end Server, REST API, API 명세서**

## 11. 데이터베이스 설계 및 **Web App** 연동

### 수행 체크리스트

- ☐ 서비스에서 저장해야 할 데이터를 정의하였다.
- ☐ **Entity**를 정의하였다.
- ☐ **Entity** 간 관계를 정의하였다.
- ☐ **ERD**를 작성하였다.
- ☐ **MySQL Database**를 생성하였다.
- ☐ **Table**을 생성하였다.
- ☐ **Primary Key/Foreign Key**를 설정하였다.
- ☐ 필요한 **SQL**을 작성하였다.
- ☐ **CRUD** 기능을 구현하였다.
- ☐ **FastAPI**와 **MySQL**을 연동하였다.
- ☐ **ORM** 사용 여부를 결정하였다.
- ☐ 실제 **Web App**에서 **DB** 데이터를 조회·저장할 수 있도록 구현하였다.

### 활용 기술 및 도구

**MySQL, SQL, SQLAlchemy, MySQL Workbench, DBeaver, FastAPI**

### 주요 산출물

**ERD, DB Schema, SQL, Database, CRUD API**

## 12. Prompt Engineering 및 LLM API 활용

### 수행 체크리스트

- ☐ 서비스에서 LLM이 수행할 역할을 정의하였다.
- ☐ 사용할 LLM을 선정하였다.
- ☐ System Prompt를 작성하였다.
- ☐ User Prompt 구조를 설계하였다.
- ☐ Role/Context/Task/Output Format을 정의하였다.
- ☐ Zero-shot/Few-shot Prompt를 적용해 보았다.
- ☐ 구조화된 JSON 응답이 필요한 경우 Output Schema를 정의하였다.
- ☐ 다양한 Prompt를 비교하였다.
- ☐ LLM 응답 품질을 평가하였다.
- ☐ Prompt를 수정하여 응답 품질을 개선하였다.
- ☐ 최종 Prompt를 문서화하였다.

### 활용 기술 및 도구

**OpenAI, Gemini, Claude 등 LLM, OpenAI API, Gemini API, Python**

### 주요 산출물

**Prompt 설계서, Prompt Template, LLM 테스트 결과**

## 13. RAG 및 Vector DB 기반 AI 서비스 구현

### 수행 체크리스트

- ☐ RAG 적용이 필요한 서비스 기능을 선정하였다.
- ☐ RAG에 사용할 문서 및 데이터를 수집하였다.
- ☐ 문서를 정제하였다.
- ☐ 문서를 적절한 크기로 **Chunking**하였다.
- ☐ **Embedding Model**을 선정하였다.
- ☐ 문서를 **Embedding**하였다.
- ☐ **Vector DB**를 구축하였다.
- ☐ **Similarity Search**를 구현하였다.
- ☐ 검색 결과를 **LLM Prompt**에 전달하였다.
- ☐ 검색 기반 답변을 생성하였다.
- ☐ 검색 결과의 적합성을 테스트하였다.
- ☐ **Hallucination**을 줄이기 위한 방법을 적용하였다.

### 활용 기술 및 도구

**Python, LangChain/LlamaIndex, OpenAI Embedding, Hugging Face, ChromaDB/FAISS/Qdrant**

### 주요 산출물

**RAG Pipeline, Vector DB, Embedding 데이터, RAG 테스트 결과**

## 14. Tools / Function Calling 및 AI 기능 확장

### 수행 체크리스트

- ☐ LLM이 외부 기능을 호출해야 하는 요구사항을 정의하였다.
- ☐ 사용할 Tool/API를 정의하였다.
- ☐ Tool의 입력 Parameter를 정의하였다.
- ☐ Function Schema를 정의하였다.
- ☐ LLM의 Tool 선택 기능을 구현하였다.
- ☐ Tool 실행 결과를 LLM에 전달하였다.
- ☐ 최종 사용자 응답을 생성하도록 구현하였다.
- ☐ 잘못된 Tool 호출 및 오류 상황을 처리하였다.
- ☐ 실제 Web App 기능과 Tool을 연결하였다.

### 활용 기술 및 도구

**Python, OpenAI Tools/Function Calling, Gemini Function Calling, LangChain, FastAPI, REST API**

### 주요 산출물

**Tool 정의서, Function Schema, AI Tool 연동 기능**

## 15. Fine-Tuning 및 AI 모델 고도화

### 수행 체크리스트

- ☐ Prompt Engineering, RAG, Fine-Tuning의 차이를 비교하였다.
- ☐ Fine-Tuning이 필요한 문제인지 판단하였다.
- ☐ Fine-Tuning용 Domain Dataset을 준비하였다.
- ☐ 데이터를 정제하였다.
- ☐ 필요한 경우 데이터 Labeling을 수행하였다.
- ☐ Fine-Tuning 대상 모델을 선정하였다.
- ☐ Fine-Tuning 환경을 구축하였다.
- ☐ Fine-Tuning을 수행하였다.
- ☐ Fine-Tuning 전/후 모델 성능을 비교하였다.
- ☐ 최종 모델을 저장하였다.
- ☐ 서비스에 Fine-Tuning 모델을 적용하였다.

### 활용 기술 및 도구

**Python, Hugging Face Transformers, PEFT, LoRA, PyTorch, Google Colab/GPU, Hugging Face Hub**

### 주요 산출물

**Fine-Tuning Dataset, Fine-Tuning 모델, 성능 비교 결과**

## 16. AI Web App 통합 및 기능 완성

### 수행 체크리스트

- ☐ React Front-end와 FastAPI Back-end를 연결하였다.
- ☐ FastAPI Back-end와 MySQL을 연결하였다.
- ☐ Back-end와 AI Server를 연결하였다.
- ☐ AI Server와 ML Model을 연결하였다.
- ☐ AI Server와 RAG를 연결하였다.
- ☐ AI Server와 LLM API를 연결하였다.
- ☐ 필요한 경우 Tools/Function Calling을 연결하였다.
- ☐ 사용자 입력부터 AI 결과까지 전체 흐름을 구현하였다.
- ☐ DB 저장 및 조회 기능을 통합하였다.
- ☐ 로그인/인증 등 필요한 사용자 기능을 통합하였다.
- ☐ 모든 핵심 MVP 기능을 하나의 Web App에서 실행하였다.
- ☐ 실제 사용자 시나리오를 처음부터 끝까지 수행하였다.

### 활용 기술

**React, FastAPI, Python, MySQL, ML Model, Vector DB, LLM API**

### 주요 산출물

통합 **AI Web App**, 통합 소스코드, 실행 가능한 **MVP**



## 17. 테스트·보안·디버깅 및 성능 개선

### 수행 체크리스트

- ☐ Front-end 기능 테스트를 수행하였다.
- ☐ API Endpoint 테스트를 수행하였다.
- ☐ Database CRUD 테스트를 수행하였다.
- ☐ AI Model Prediction 테스트를 수행하였다.
- ☐ RAG 검색 결과를 테스트하였다.
- ☐ LLM 응답 품질을 테스트하였다.
- ☐ 정상 입력/비정상 입력을 모두 테스트하였다.
- ☐ 오류 및 예외처리를 확인하였다.
- ☐ Prompt Injection 등 AI 보안 위험을 점검하였다.
- ☐ API Key 및 개인정보가 외부에 노출되지 않는지 확인하였다.
- ☐ GitHub에 Secret 정보가 올라가지 않았는지 확인하였다.
- ☐ API 응답속도를 측정하였다.
- ☐ AI 응답속도 및 비용을 점검하였다.
- ☐ 발견된 오류를 수정하였다.
- ☐ 수정 후 Regression Test를 수행하였다.

### 활용 도구

**Postman, Swagger, pytest, Chrome DevTools, VS Code Debugger, GitHub Issues, AWS CloudWatch**

### 주요 산출물

테스트 시나리오, 테스트 결과서, **Bug List**, 개선 결과

## 18. Nginx·AWS EC2 기반 배포 및 서비스 운영

### 수행 체크리스트

- ☐ AWS 계정을 준비하였다.
- ☐ EC2 Instance를 생성하였다.
- ☐ Ubuntu/Linux 환경을 구성하였다.
- ☐ Security Group을 설정하였다.
- ☐ SSH로 EC2에 접속하였다.
- ☐ GitHub Repository의 프로젝트를 서버에 배포하였다.
- ☐ Python 및 Node.js 환경을 서버에 구성하였다.
- ☐ React Build 파일을 생성하였다.
- ☐ FastAPI Server를 실행하였다.
- ☐ AI Server를 실행하였다.
- ☐ MySQL 또는 Database 연결환경을 구성하였다.
- ☐ Nginx를 설치하였다.
- ☐ / → React 경로를 설정하였다.
- ☐ /api/\* → Back-end 경로를 설정하였다.
- ☐ /ai/\* → AI Server 경로를 설정하였다.
- ☐ Nginx Reverse Proxy를 구성하였다.
- ☐ 환경변수 및 API Key를 안전하게 설정하였다.
- ☐ 외부 Web Browser에서 서비스에 접속하였다.
- ☐ 전체 Web App 기능을 실제 서버에서 테스트하였다.
- ☐ Server Log 및 오류 로그를 확인하였다.
- ☐ 서비스 중단 및 재실행 방법을 확인하였다.

### 활용 기술 및 도구

**AWS EC2, Ubuntu Linux, SSH, Nginx, Git/GitHub, Python, Node.js, Docker(선택), CloudWatch**

### 주요 산출물

배포된 **Web App**, 배포 **URL**, 서버 구성 문서, 운영 매뉴얼

# 프로젝트 최종 결과물 체크리스트

## 기획

- ☐ 문제정의서
- ☐ 사용자 분석
- ☐ 서비스 기획서
- ☐ 요구사항 정의서
- ☐ MVP 정의서
- ☐ 프로젝트 일정 및 역할표

## UX/UI

- ☐ Sitemap
- ☐ User Flow
- ☐ Wireframe
- ☐ Figma Prototype
- ☐ 최종 UI

## 시스템 설계

- ☐ 시스템 아키텍처
- ☐ 데이터 흐름도
- ☐ ERD
- ☐ API 명세서
- ☐ AI 처리 구조도

## 데이터/AI

- ☐ 데이터셋
- ☐ 데이터 분석 결과
- ☐ 전처리 코드
- ☐ AI 모델
- ☐ 모델 평가 결과
- ☐ Prediction API
- ☐ Prompt 설계서
- ☐ RAG Pipeline
- ☐ Vector DB
- ☐ Fine-Tuning 결과(선택)

## Web App

- ☐ React Front-end
- ☐ FastAPI Back-end
- ☐ MySQL Database
- ☐ AI Server
- ☐ LLM API 연동

- ☐ 전체 시스템 통합

## 테스트/배포

- ☐ 테스트 시나리오
- ☐ 테스트 결과서
- ☐ Bug 수정
- ☐ AWS EC2 배포
- ☐ Nginx Reverse Proxy
- ☐ 최종 서비스 URL
- ☐ 운영 매뉴얼

## 발표/포트폴리오

- ☐ GitHub Repository
- ☐ README
- ☐ 프로젝트 포트폴리오
- ☐ Demo 영상
- ☐ 제안서
- ☐ PT 자료
- ☐ 최종 Demo
- ☐ 최종 발표

# 프로젝트 최종 완료 기준 자가 점검

프로젝트는 단순히 소스코드가 완성된 상태가 아니라 다음 조건을 충족하는 것을 최종 완료 기준으로 한다.

- ☐ 실제 해결하고자 하는 비즈니스/사용자 문제가 정의되어 있다.
- ☐ 문제를 해결하기 위한 **AI Web App** 서비스가 기획되어 있다.
- ☐ Figma 등을 활용한 **UX/UI** 설계 및 **Prototype**이 존재한다.
- ☐ **React + FastAPI + Database** 기반 Web App이 동작한다.
- ☐ **AI** 모델 또는 **LLM**이 서비스에 실제 적용되어 있다.
- ☐ 필요한 경우 **RAG/Vector DB**가 적용되어 있다.
- ☐ 필요한 경우 **Fine-Tuning** 또는 모델 고도화를 수행하였다.
- ☐ **Front-end → Back-end → AI → DB/LLM**의 전체 데이터 흐름이 정상적으로 동작한다.
- ☐ 실제 사용자가 수행할 수 있는 핵심 사용자 시나리오가 완성되어 있다.
- ☐ 테스트 및 오류 수정을 완료하였다.
- ☐ **AWS EC2 + Nginx** 환경에 배포되어 외부에서 접속할 수 있다.
- ☐ 프로젝트의 개발 과정과 결과물을 **GitHub** 및 포트폴리오로 정리하였다.
- ☐ 서비스의 필요성과 기술적 구현 내용을 제안 **PT**를 통해 설명할 수 있다.